

uSEQ: A LISP-y Modular Sequencer for Eurorack with a Livecodable Microcontroller

Dimitris Kyriakoudis

University of Sussex / Experimental Music Technologies (Emute) Lab

D.Kyriakoudis@sussex.ac.uk

Chris Kiefer

University of Sussex / Experimental Music Technologies (Emute) Lab

C.Kiefer@sussex.ac.uk

ABSTRACT

uSEQ is a new livecodable sequencer module for the Eurorack modular synthesiser ecosystem. It draws inspiration from both the practices of live coding and hardware modular synthesis, aiming to examine and combine their respective ergodynamic strengths and weaknesses into a new, hybrid practice. It embeds *uLisp*, a tiny interpreter for the general-purpose language with a small-enough footprint to run even on inexpensive microcontrollers. The interpreter’s REPL has been modified for the purposes of live coding, and its reader has been “hacked” to the limited resources of the sub-£5 microcontroller at the heart of the module. On top of the Lisp general-purpose language, a minimal DSL layer has been designed to take full advantage of the linguistic flexibility of the underlying language. Simultaneously, the semantics of this DSL tries to stay close to one of the central philosophies of modular synthesis: everything is a signal, and sequences are no exception. Its open-source firmware and design files (PCB and 3D-printable faceplate), with an approximate total cost of £20 in parts, *uSEQ* is highly DIY-friendly and can be built as a weekend project. It provides multiple input and output interfaces to the rest of the Eurorack ecosystem, and can be controlled from a mobile phone, tablet, or any (micro)computer with a USB port. The prototyping of this module is a practical exploration into the ways in which the ergonomic and ergodynamic tensions between the practices of livecoding and modular synthesis can be successfully addressed when bridging the two worlds, following a practice-led, research-through-design approach.

1 Introduction: Livecoding and Modular Synthesis

Many livecoders experiment with extending their instrument beyond the typical laptop and text editor setup, to include elements such as hardware controllers and outboard synthesisers. Hardware modular synthesisers are a compelling option; they offer a contrasting and highly complementary alternative to livecoding in terms of interactivity, sound, and visual and sonic aesthetics. But how compatible are these two domains, and how can we build effective bridges between them?

In this section, we outline our motivations and methods for attempting to bridge the two worlds of livecoding and modular synthesis. We discuss the ways in which the two instrumental practices differ, their in-between ergonomic and ergodynamic tensions in matters of cognition and embodiment, as well as the ways in which they could complement and extend each other. Finally, we discuss our research question, which sets the context for the rest of the work presented in this paper.

1.1 Tensions

Most livecoding environments offer practically-infinite reconfigurability and replicability of their constituent elements: at any given moment, one is only a handful of keystrokes away from conjuring and (re)patching as many oscillators, envelopes, and sequencers as one’s computer can handle—the number and configuration need not be predetermined and can vary drastically over time, even during a performance. On the contrary, modular synthesis requires that the number, kind, and placement of available elements be fixed in advance, during the practice and preparation stage for a performance. Modules are often selected, acquired, and brought together in a case months in advance of a performance, to allow for plenty of time to become intimately familiar with their design, controls, behaviours, and peculiarities—sonic and otherwise—especially in the ways they interact with other modules.

Whether writing a line of code or patching a modular system, fundamentally both activities are very similar (Hutchins 2015): the player is (re)configuring a computational system to make sound and/or music. Yet, both approaches have very different ergonomics (Magnusson 2019): they have a specific *feel* to them, guiding us into particular ways of making music due to the opportunities, limitations, and designs of their internals and their interfaces alike. Simultaneously, both practices encourage a highly reactive form of creativity; the sheer complexity of the system’s internal configuration and the resulting interaction of its constituents at any one moment makes it so that actions often have unintended and unexpected consequences, which the player is encouraged to react to, embrace, and play with in their performance.

Modular synthesis presents itself as a ‘rough analogue computation system’, through what we might call ‘scientific instrumentation paradigm’ interfaces. These interfaces offer tactile control over digital and analog circuitry, and are fixed in their design, physical form, and function over time. The values of parameters in the computational process are usually reflected physically on the surface of the instrument, in the form of flashing LED lights, marked dial and fader positions, and colourful patch cables, often leading to an overwhelming degree of visual complexity. This contrasts with the extremely precise and predominantly linguistic nature of complexity in livecoding, where the almost-exclusively textual interfaces require significant attention and effort to semantically parse the code and keep up with its changing form throughout a live set. At the same time, code naturally tends to try and hide away informational complexity by means of stacking compounding layers of linguistic abstraction, thus being more demanding in terms of short and long-term working memory in order to maintain an accurate model of the code in one’s mind over time.

Livecoding’s strictness around correctness, fault intolerance, hard requirement for precision, and computational accuracy starkly contrasts with modular’s more nonlinear, imprecise, and fault-tolerant approach to computation and the emergent and loose style of play it promotes. While a modular synthesis interface may visually reflect the current values of most (or even all) of the parameters, these visual readings are mostly rough and imprecise estimates; one cannot read the exact frequency of an oscillator or filter, or the precise attack and decay times of an envelope, just by glancing at a dial with infrequent or no markings. On the other hand, in livecoding the player is asked to enter and interact with precise and unambiguous numerical values, encouraging them to build a more intimate creative relationship with the numbers over the course of their practice, as opposed to a modular instrument’s explicit imprecision and how it promotes a ‘play with your ears, not your eyes’ approach to parameter tweaking. Furthermore, code is rather strict in its approach to correctness and its tolerance to mistakes; it will not hesitate to refuse a change and display cryptic compiler errors, which offer little explanation as to how to fix the problem—or what the problem even is in the first place. On the other hand, the hardware modular paradigm is a lot more forgiving: by virtue of everything being an electronic signal, even the most unconventional of patchings can produce *some* result without crashing the system—the circuits are designed to be “childproof” and safely accept any and all patching combinations, allowing plenty of room for experimentation without fear of crashes or damage to the instrument.

This tension is also reflected on the aesthetics of the two practices: livecoded sound and music has been accurately and precisely calculated, with no ambiguity or computational noise, whereas sound produced through analogue computation can be much more sensitive to accidental instability and imprecision; many analogue oscillators and filters are infamously temperature sensitive.

1.2 Bidirectional Extensions

A livecoding approach to modular synthesis could offer novel ways to use these hardware computation systems for music-making. For example, some programming languages display an impressive capacity for combining high degrees of *information complexity expressivity* (how easily and concisely one can express complex patterns) with remarkable visuospatial and cognitive terseness. As modular synthesists will attest, space is a premium in modular synthesis setups, and terseness of representation and interfaces is valued highly in the design of modules. This expressivity could also inspire modular synthesists to compose musical structures that would be otherwise impractical to represent and unwieldy to control in the physical confines of a hardware interface. As another example, the arbitrary flexibilities presented by code’s fluidity and customisability could provide, when desired, nuanced rhythmic and compositional structures that break free from the confines of the extreme ends of the spectrum: the restrictive rigidity of perfect repetitiveness or the unwieldy randomness; both of which many Eurorack sequencers seem to gravitate towards.

In the opposite direction, the rich and diverse timbral ecosystem of modular synthesis can provide a welcome breath of fresh sonic air to the livecoding practice, offering new voices to the computational algorithms that go beyond what the purely-digital can achieve; the nonlinearities and other intricacies of many analogue circuits have long been sought-after in the world of digital sound synthesis and processing. Not only can the livecoder’s sonic palette be extended, but their embodied control over their sound too: many modules offer effective and even novel methods of interacting with sound through one’s body, for example in the form of dials, joysticks, or touch-sensitive capacitive surfaces. Livecoders may gladly welcome such degree of embodied interaction with their sound as an alternative to the ubiquitous binary switch interface of the traditional computer keyboard.

1.3 Reconciliation

We have discussed the musical and workflow differences between the two practices, the tensions that arise when considering them in parallel, as well as the ways in which one could potentially extend the other. We now ask: How can we address the ergonomic and ergodynamic tensions between the two practices and bridge the two worlds successfully?

Imagine a system where a livecoding instrument is being used to trigger and guide sequences on a modular synthesiser. The player might suffer from fragmenting their attention between two very different kinds of cognitive artifacts (Norman 1991) and styles of logic: analogue computation, with its continuous parameter space and respectively continuous tactile controls, versus coding, with its perfectly discrete states and similarly discrete binary input interface (the computer keyboard). They might also struggle with the differences in embodiment between the two interfaces: the modular is timely, immediate, and gradually-changing (a performer has, at most, only two hands available to execute changes at any one moment), where the code interface blurs the temporal link between changes in code and changes in the produced music and stretches time between evaluations, often leading to the accumulation of multiple changes and their simultaneous execution to dramatic effect. The modular offers hands-on, direct, and sensitive physical control over a largely continuous parameter space, where the computer offers an indirect button interface to a complex system of logic comprising the text editor, the livecoding language, and the algorithmic thought itself. More practically, the physical distance between the computer and modular system, especially when using a desktop computer system, requires that the player's body—and therefore attention—be constantly shifting from one to the other in order to operate both with any degree of timeliness, which can be rather disruptive to one's workflow and capacity to concentrate. Kiefer discusses the challenges in balancing *between code and controllerism* (Kiefer 2018) in performance, reflecting on the cognitive mismatch between the styles of control.

We can look to the research of Armstrong (Armstrong 2006) on enactive musical instruments to further highlight the differences between these styles of musical interaction: the *computer-as-it-comes* challenges or breaks the conditions of timeliness, multimodality and engagement that support rich embodied interaction. The combination of conventional livecoding environment and modular synthesis might possess an internal tension or *ergodynamic dissonance* that challenges the player. It follows that the challenge in our research was in how to try and reduce this dissonance by designing a livecoding system that tried to fit within the ergonomics of modular synthesis. In short, *how can we combine the flexibility of livecoding with the richness and tactility of modular synthesis in a useful way, by reducing this dissonance between systems and applying each to its respective strengths?*

In addition to this question, we further narrowed down the scope of our research to attempt to follow a minimalist, affordable and accessible DIY approach. We see this as being a common spirit of both the modular synthesis and livecoding approaches to instrument- and music-making, where it's not only possible but also encouraged to build your own instruments, and where many projects follow an open source ethic which in turn encourages engagement and creativity (Mellis and Buechley 2011).

The result of this project is *uSEQ*, a new eurorack sequencing module, which costs approximately £20 to build, and can be livecoded from any computing device that runs a serial terminal, including tablets, mobile phones, micro-computers like the Raspberry Pi, or other eurorack modules. The module runs entirely on a Raspberry Pi Pico, a widely-available sub-£5 microcontroller, which implements a signal processing engine that is programmed and programmable using a variant of the Lisp programming language.

2 Context

Before describing the design of this module in more detail, we will first explore the background artistic and research context. Here we will outline other projects that form the technical and aesthetic basis of the approach we have ourselves followed when designing our new livecodeable module.

2.1 Sequencing in Livecoding and Modular Synthesis

Sequencing is innate to modular systems in that explicit sequencer modules are not required; any signal in the system can be re-interpreted as a control voltage (CV), and many can act as triggers (impulse-like momentary spike in voltage) and gates (separate 'on' and 'off' events, distinct in time). Beyond this barebones approach, there are a multitude of purpose-built sequencer modules available, from conventional step sequencers to those using more esoteric approaches such as cartesian, euclidean, or other algorithmic and probabilistic methods. One might observe that many of these sequencers are highly specific instruments: they focus on a particular method of sequencing, which often limits the possible range of musical outcomes to only those representable by the module's central algorithm. Some modules offer one-to-one interface mappings, sacrificing flexibility, upgradeability, and "case real estate" for direct control. Others,

in their attempt to provide more features, end up having to hide many of them “under the hood”: as options in nested menus that are navigated via small displays and buttons or dials. Their fixed interfaces make those features hard to reach and potentially challenging and impractical to use, which in turn greatly affects the module’s ergonomics and ergodynamics in both practice and play. It follows that livecoded sequencers could make a useful addition to modular sequencing, bearing in mind the aforementioned challenges with ergodynamic dissonance.

The first and probably easiest step into livecoding a modular would be with a MIDI connection from a traditional laptop livecoding system. There are, however, some serious disadvantages with this approach, not least the fact that the MIDI protocol has an infamously low resolution at 7 bits of information. Considering that the source (the modern laptop) typically produces 16-bit audio signals and that the destination (the analogue modular synthesiser) is capable of utilising a practically-infinite resolution, this seems like a rather wasteful translation. This is, of course, unsurprising when considering its status as one of the oldest digital protocols still in widespread use today, more than 40 years after its creation; yet it begs the question of why it should dictate the quality of communications between two cutting-edge modern platforms with much higher capacities. Secondly, MIDI processing in livecoding environments tends towards discrete systems concerned with the symbolic representation of notes and rhythms, breaking the paradigm of signal impartiality in modular synthesis and creating a cognitive artifact dissonance that can fragment and degrade the player’s attention, comprehension, and consequently experience.

The next step up is to interface the two system using continuous signals, generated and fully formed in the computer and transduced to the analog domain through connected DACs. For example, McKemie (McKemie 2015) shows how to livecode a modular using Chuck, connected through DC-coupled audio using an Expert Sleepers module. This approach is a better match between the mechanics of digital and analogue audio systems, but doesn’t solve the difference between interaction styles of the two systems. It also doesn’t successfully integrate into the physical structure of a modular synthesiser case, instead requiring an additional external device besides the laptop and the synthesiser.

An alternative approach is to embed the livecoding engine within the modular system in the form of a compatible module. There’s certainly technology that can facilitate this already: modules such as SALT (Bela), Daisy Patch (Electrosmith) or OWL (Rebel Technology) should have more than enough computational resources to host a livecoding system. Two examples of modules that have livecoding engines are Drymonitis’s livecodeable 3dPdModular *Code* system. (Drymonitis 2022) and Monome’s Teletype (Monome 2022). Both of these modules have embedded screens, with control taken from a keyboard connected to the module. The *Teletype* offers the user a domain specific language (DSL) for pattern sequencing of triggers and gates. The *Code* offers a Python interface, interpreted as a Pure Data patch. The author notes the flexibility of the livecoding approach, along with challenges from the small screen and in integrating with architecture of the *3dPdModular* system.

Taking inspiration from these modules, our project explores this embedded module approach further, with a focus on dynamic, configurable language and API, and lightweight, low-cost DIY technology, using a microcontroller. Notably, we explicitly choose to label our module as a sequencer, assigning to it a fixed and well-defined role in the context of a modular system and stripping it of its potential to be an “anything goes” DSP system, which we feel is far from an instrument and closer to an instrument-building platform. As active livecoders and digital luthiers ourselves, we felt the need to prevent this module from becoming just a mere continuation of our development workflow that happens to spill all over our modular worlds, instead wishing for the feel of a more well-defined instrument than what can be found when presented with a computer that can run any SuperCollider or Pure Data patch.

2.2 Livecoding with Embedded Computing

Microcontrollers might not be the first thought when it comes to livecoding: these devices tend to have very limited memory, relatively low CPU power, and lack important computing resources like floating point processors or audio-grade inputs and outputs. One might often find them in modules, but preloaded with fixed and closed firmware. There have been some explorations of microcontrollers in the livecoding community: Fredrik Olofsson’s piece *chain reaction* (Olofsson 2015) at ICLC 2015 approached livecoding by alternating between flashing firmware between Arduino boards. Kiefer’s *DJ FPGA* performance at ICLC 2020 followed this approach, mixing between two FPGA boards with a DJ mixer.

2.3 Minimalist Languages and Livecoding Systems

While it is essential for any programmer to have a language that can closely model the problem domain of the task at hand and remain free of unnecessary and irrelevant burdens, complications, and distractions, this need only becomes more pressing when considering the real-time cognitive and physical constraints and sensitivities of a live coding practice.

Minimalism in the format representation does not necessarily equate with minimalism in the patterns that can be represented. Notably, TidalCycles (McLean 2014) embeds a mini-DSL with simple syntactic rules, inspired by the notation

used to represent rhythms in the Carnatic music tradition—it therefore comes as no surprise that extremely complex rhythmic patterns can be easily represented and modified with the usage of very few characters.

Another example of minimalism in live coding environments can be found in Orca, the work of artist and technologist collective 100rabbits. Their motivations for minimalism extend towards the ethics of being able to create and maintain one’s own tools, as well as minimising their energy and carbon footprint (100rabbits, n.d.), concerns which may well apply to the DIY modular synthesisist too.

But minimalism does not limit itself to the appearance of a language, as syntax is one but many of a language’s core dimensions. Ontological and semantic minimalism is arguably just as important; within the temporal pressure of an improvised live performance, the language’s design should aim to reduce usage of inefficient and unnecessarily complicated ontological distinctions and semantic frivolities to a minimum; as low as possible without severely impacting the language’s expressive power.

Modular synthesis’ own paradigm is one of true unification of representation: everything is represented and operated upon as electrical connections carrying over-time fluctuations in voltage. Similarly, many livecoding languages attempt to unify the underlying representation of seemingly distinct entities, such as sequencers, envelopes, and LFOs. TidalCycles’ ontology of “everything is a Pattern of time” enables a certain freedom of modular patching (mediated through its parent language Haskell’s strict static type system), while similarly TimeLines (Kyriakoudis 2018) and its “everything is a numeric function of time” approach reduces all entities to mere streams of time-varying numbers. This approach is not only minimal but also powerful, stripping away unnecessary ontologies and metaphors and exposing an underlying 1-to-1 mapping between the algebra of numbers and that of sonic and musical signals and structures.

3 The uSEQ Module

We now describe the design and implementation of the *uSEQ* module, starting with the design requirements, followed by the hardware configuration and software implementation.

3.1 Design Requirements

The requirements we decided upon respond to our research questions around ergonomics and ergodynamics of the two categories of instruments, as well as a desire to support an affordable and open-source DIY approach—a core part of both the livecoding and modular synthesis cultures:

1. The system should use low-cost hardware that is widely accessible.
2. The module should be able to be built by hobbyists as a DIY project, in no more than the span of a weekend.
3. The instrument should aim to blend the ergodynamics of modular synthesis and livecoding, finding a common language that will not unnecessarily fragment the player’s attention and further burden their cognitive and operational load.
4. The module should integrate with the rest of the system by accepting inputs from other modules and providing CV and gate/trigger outputs, with a better resolution and fidelity than MIDI solutions.
5. The livecoding system should be flexible and extensible by the user, so that they can customise the module to match any unconventional or idiosyncratic requirements.

3.2 Hardware Design Decisions

When designing the hardware architecture of the module, using a microcontroller as the core element was an obvious choice. Microcontrollers are highly affordable, have extremely short boot-up times, a surprisingly strong ratio of performance over size and cost, and often come with a wide variety of general-purpose IO capabilities that can be used to interface with the rest of the Eurorack world. They also require relatively little power, an often limited resource in modular racks, compared to single board computers such as the Raspberry Pi. It is therefore no surprise that microcontrollers have established themselves firmly into the Eurorack world, powering an increasingly large percentage of modules, both fully digital and hybrid.

Performance-wise, our goal for the livecodability capacity of the sequencer dictated that the microcontroller should have enough resources to be able to embed an interpreter for a general-purpose language, powerful and flexible enough so that we could seamlessly embed our DSL interface within it.

We chose the Raspberry Pi Pico for our module. This is a board with a modern RP2040 microprocessor and a reasonable amount of RAM and CPU power, plenty of flexible IO, and a comparatively low cost. It also features a dual-core CPU architecture, which could allow for better performance through the parallel execution of code.



Figure 1: *First prototype module, with a 3D-printed panel (using the older working name pSeq), housed in a Eurorack case with other modules.*

3.2.1 Form and Connectivity

With the aim of a low-cost module, and to aid extensibility, we decided on a small format at 6hp wide. *uSEQ* has two digital inputs to enable control and synchronisation from other modules. It also has four digital (binary high/low) outputs, and two continuous outputs; these use the Pico's PWM system with an above-hearing-range carrier frequency of 30kHz to output signals in 0-5V range with 11-bit resolution—a significant improvement over MIDI's 7 bits. LEDs accompany each input and output, offering a visual indication of its moment-by-moment state. *uSEQ* also has a selection of physical controls: two momentary switches, two toggle switches, and a digital rotary encoder with a momentary switch. There is a pin-accessible reset switch, in case the player crashes the microcontroller. Notable is the exclusion of continuous inputs and physical continuous controls; we omitted these to minimise the complexity of the prototype, but they could easily be included in future iterations.

We decided that the module would not have its own display; instead the player would connect to the module using another computer. Space is a scarce and precious commodity in the modular world; while large interfaces can provide better control and higher versatility, they can also quickly increase the size and cost of production of an instrument, competing for space with all other modules. There is a constant trade-off to be made between how much space and money a module is going to cost and how usable and practical it can be. On the other hand, code can be easily customised and shaped to fit one's needs, and a device such as a tablet or a mobile phone may offer enough representational space (considering code's upper bound for terseness) while simultaneously not competing with other modules for space in a rack. A mobile device is also easy for one to hold on their person or to position close to a modular system, as opposed to a desktop or even laptop computer.

3.2.2 Circuit Design

The schematic is shown in Appendix 1. The number and complexity of components are kept to a minimum to keep costs low. Most of the circuit deals with connectivity to the Pi Pico. A CD4504 hex voltage level shifter converts the Pico's 3.3v outputs to 5V. 'Analogue' PWM outputs are smoothed using simple resistor-capacitor filters. Digital inputs are managed by a simple bipolar transistor circuit that clips voltage to a safe range for the microcontroller. The module needs only 5V power.

3.3 Software

This section elaborates on our design and implementation choices in prototyping this module, as well as the reasoning behind them and the ways in which they are aligned with our research question and goals.

3.3.1 Choice of Language and Implementation

In accordance to our goals, we needed a general-purpose language interpreter that would have minimal hardware requirements while also providing a flexible and powerful (meta)programming environment. We also wanted to enable the user to reach past the DSL layer and tap into the full power of the underlying general-purpose language when and as needed, thus allowing them to build custom and more advanced algorithmic structures—or different DSLs all together. We chose to embed our livecoding DSL inside the Lisp programming language through the *uLisp* interpreter implementation, for the following reasons:

1. **Low hardware requirements:** *uLisp* was created specifically for embedded environments with limited computational resources such as microcontrollers, and therefore fits and runs perfectly on the Raspberry Pi Pico.
2. **Linguistic, syntactic, and notional minimalism:** Lisp is (in)famous for its extremely minimal syntax, which mostly consists of parentheses that denote the grouping of elements into lists (the name LISP itself stands for *LISt Processing*). It also has a minimal core, which lends itself perfectly to lean and lightweight implementations, precisely like *uLisp*, which implements a fully-functional subset of the otherwise extremely comprehensive Common Lisp language specification.
3. **Homoiconic metaprogramming:** Homoiconicity is a trait of languages whose source code is no different to the regular data structures that the programmer deals with and that the language itself can understand, generate, process, and evaluate (McIlroy 1960). Lisp is perhaps the most famous of those, with its powerful macro system being an integral part and consequence of the language’s core design. It allows the programmer to utilise the full power of the language to generate and modify code, both at compile-time but also during runtime execution, which translates to extremely powerful metaprogramming capabilities at no extra cost or complexity. Compared to the metaprogramming capabilities of other languages, it has been famously said that (paraphrased) “C++ templates are to Common Lisp macros what IRS tax forms are to poetry.” (Schafmeister 2018). This feature alone is what makes Lisp stand out over other candidates, such as the Micro-Python implementation which can also run on the Pico’s hardware.
4. **Maturity of the language:** Common Lisp is an ANSI-standardised language and one of the oldest languages still in active use today, with multiple high-quality educational resources available in print and freely online. Its design has been battle-tested and crystallised through decades of use in multiple industries, managing to stay relevant over the decades precisely because of its ability to adapt to new domains and tasks through its capacity to linguistically transform itself.

3.3.2 Signal Engine Design and Implementation

In accordance with the ‘linear algebra as a DSL’ approach we chose to follow, time and temporal structure is represented through a series of numeric variables that are closely related. We also thought this approach to be simple enough to implement and fall realistically within the computational performance capabilities of our microcontroller, as opposed to more complex timing execution models.

This approach mostly falls under the functional programming paradigm, although the user can choose to make use of mutable state and imperative-style code if they wish; the limited computational resources of a microcontroller are unfortunately not entirely well-suited to the space and time demands of the purely-functional style of programming; although in our tests this has not proved to be a practical issue.

Firstly, the system defines a number of built-in core variables: `time`, `reset-time`, and `t`, all of them in milliseconds. Their roles are as follows: `time` always represents the absolute amount of time that has elapsed since the module was powered on, in milliseconds, whereas `t` represents the amount of time that has elapsed since the last reset of the module’s internal transport (which can either be triggered by the user or by a trigger in one of the two digital inputs):
$$t = \text{time} - \text{reset-time}.$$

The user can then specify a tempo (in BPM) and meter structure; these are then used to calculate two core variables, `beatDur` and `barDur`, both of which are constant values that represent the duration of a beat and a bar respectively, in milliseconds. Following, these are used to derive four relative values: `beat`, `bar`, `beatNum`, and `barNum` which represent the ever-increasing current beat and bar values in floating-point and integer representations respectively.

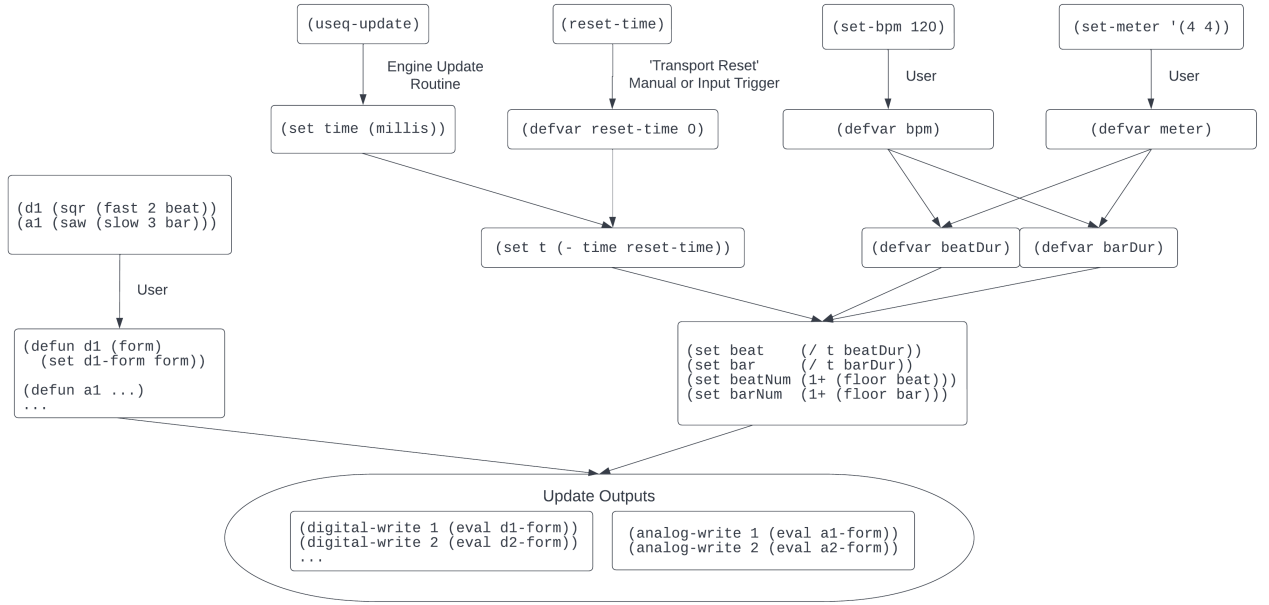


Figure 2: The DSL's timing model.

In order to update these variables, the module's update routine starts by setting time to the result of calling the microcontroller's `millis` function, followed by the successive updating of all dependent variables. Once that process has completed, the engine will then execute the last-stored forms associated with each of the 4 inputs: `d1` through `d4` and `a1` & `a2`, for digital and analog outputs respectively. These forms are plain Lisp lists which represent a computation graph that should be executed every iteration of the update loop; the higher the rate of execution, the higher the sampling resolution. The way that change is produced in the output is by providing a form which uses the value of at least one of the built-in time-tracking variables; for example, `(d1 (sqr (fast 2 beat)))` will cause a square wave with a period of 2 beats to be output from the first digital output. Executing the update forms is a simple case of calling Lisp's built-in `EVAL` function on them, which requires that the time-tracking variables have already been updated.

The values of the module's digital inputs, as well as the controllers, are read once at the beginning of each iteration of the update loop and stored in globally-accessible constant variables, ready to be accessed in the body of any of the outputs' update forms.

Figure 2 outlines the overall structure of the signal engine's timing model and update routine, as well as all the input points where the user can trigger changes to the module's global timing transport.

3.3.3 DSL Design

We have chosen the aforementioned 'linear algebra as a DSL' approach for the DSL that powers *uSEQ*. We believe it offers an excellent balance of minimalism, general familiarity, modularity, compositionality, and expressive power. Equally importantly, it is also perfectly aligned with the 'everything is a time-varying signal' approach of modular synthesis, thus reducing cognitive metaphor and artifact dissonance.

Following from functional paradigm observed in the design and implementation of the module's signal engine, it follows that the user is expected to produce sequences by writing forms of code, one for each of the available outputs, which somehow reference at least one of the system's time-tracking variables to produce fluctuations in voltage over time.

Our DSL, following the approaches of *TimeLines* and *Sema* (Bernardo, Kiefer, and Magnusson 2020), opts for pattern-generating functions that make use of explicit 'phasor drivers': signals that, by virtue of their over-time fluctuation within the normalised range of $[0 \text{ to } 1]$, drive the relative phase of various other signals and patterns in a compoundable fashion.

To illustrate, consider the public API functions `sqr` and `sine`. In contrast with *TidalCycles*' implicit flow of time, which allows these functions to need zero arguments to produce their patterns, these implementations expect an explicit phasor argument, expected to be a normalised signal. The function `sqr` will simply produce the number 1 while its argument is within the range of $[0, 0.5)$ and the number 0 otherwise, while `sine` will unsurprisingly return a full cycle of sine wave when driven through its phase domain. When driven by a linear phasor, for example the `beat` or `bar`

phasors provided by the API, then the resulting output signals will be a square and a sine wave oscillators respectively, with a period equal to that of the phasor's.

Of course, the linearity of the beat and bar phasors can be tampered with, for example by raising them to an exponent that's either higher or lower than 1 using the `pow` function, or by otherwise scaling and offsetting them. This will result in respectively-skewed versions of the square and sine LFOs. While the modules expect the phasor to be normalised within the `[0, 1]` range, they will also accept any other value and will simply wrap them around by applying a 'modulo 1' operation. This is in accordance with the spirit of the hardware modular synthesis paradigm, where modules are encouraged to accept as wide a range of inputs as possible to maximise fault tolerance across the system.

Temporal manipulation of control structures happens through the `fast` and `slow` API functions, which operate similarly to `TidalCycles` or `TimeLines`: their first argument is a number that represents the factor by which the time domain of their second argument should be compressed or stretched, respectively. In conjunction with phasors like `t`, `beat`, and `bar`, `fast` and `slow` are aimed to be the core tools for temporal structure and manipulation in the *uSEQ* livecoder's arsenal.

While `fast` can be implemented relatively easily by scaling the phasor by the factor and then applying a 'modulo 1' operation on the result, the only realistic implementation we could think of a function such a `slow` was to design the system so that phasors are not normalised by themselves but are instead implicitly normalised by each function that has such a requirement. This differs from the implementation of the equivalent `slow` function in systems such as `TidalCycles` or `TimeLines`; there, signals are fully-featured function-like datatypes and symbolic representations respectively, whereas in *uSeq*'s DSL they are regular and opaque functions that return numbers. This is why phasors such as `beat` and `bar` are ever-increasing in our design, and why every function that requires a normalised phasor will take it upon itself to enforce that behaviour upon its inputs by applying a 'modulo 1' operation.

Another core API function is `fromList`, which takes a list of elements and a normalised phasor and returns the element in the position indicated by the phasor, relative to the list's length. For example, in the form `(fromList '(1 2 3 4) _)`, numbers in the range of `[0, 0.25]` will return the value 1, the range `[0.25, 0.5]` will return the number 2 etc.

Forms such as `(range 0.4 0.8 <signal>)` or `(* bar01 (sine (fast 8 beat)))` may be already familiar to those with some experience in live coding: the first will scale the output range of a signal to be within the bounds specified, while the second will gradually and linearly fade-in a sine wave LFO over the course of a bar. In the second expression, `bar01` is another built-in timing variable, equivalent to `(mod bar 1)`, useful for situations where the receiving function—in this case `+`—has no way of knowing whether its argument should be wrapped back around after it reaches a value of 1 or not. This is a tradeoff that our DSL has to make, say in comparison to `TidalCycles` or `TimeLines`, in exchange for its radical simplicity in implementation.

As an example of a more complex expression, the following assigns a square wave to the first digital output, modulating its speed (from the baseline of once per bar) by iterating through a series of speed factor values over the span of 4 bars. The first three values are constant, while the last value is itself a time-varying signal, alternating between the values 16 and 9 every beat.

```
(d1 (sqr (fast
  (fromList '(3 6 8 (fromList '(16 9) beat))
    (slow 4 bar))
  bar)))
```

A thorough tutorial and many example code snippets can be found at the module's Github repository¹.

3.3.4 Challenges

One of the main challenges we faced with using *uLisp* was when attempting to utilise the Raspberry Pi Pico's dual-core CPU architecture, by evaluating the REPL loop (which typically blocks execution while waiting for user input) on one core and the signal engine update loop on the other, which would separate execution concerns and lead to an improvement in the module's responsiveness and sample-rate stability. After multiple failed or semi-successful attempts, we discovered that *uLisp* does not officially support multicore execution on the Raspberry Pi Pico yet.

We were then faced with a decision to make: we could either attempt to extend the interpreter's implementation to support multicore execution, or we could attempt to re-arrange the implementation of the REPL to fit both loops on one core. We chose the later option for this first prototype, which was implemented by modifying the REPL to perform the following loop: 1. Perform garbage collection (in order to maximise available memory before the start of each update

¹<https://github.com/chriskiefer/ulisp-arm-livecode>

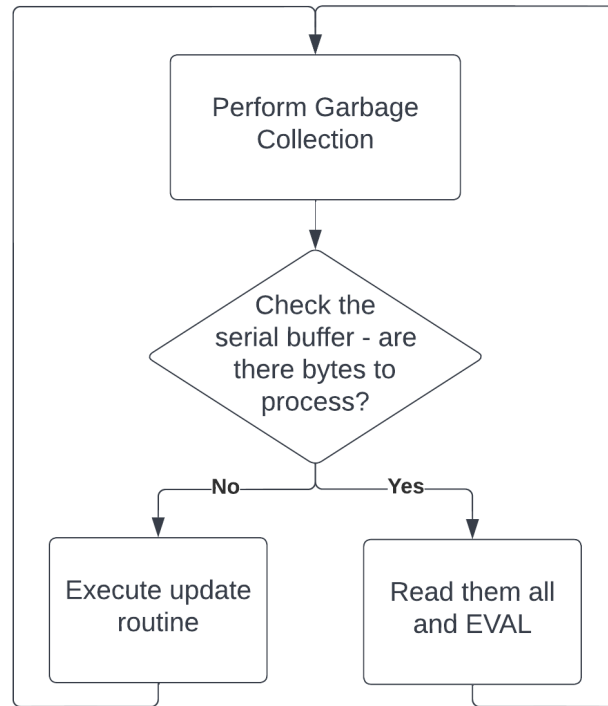


Figure 3: The interpreter’s modified REPL loop.

loop). 2. Check if there are any bytes in the serial connection buffer, which represent code that the user has sent to the module for execution. 3. If there are, read and *EVAL* expressions until the buffer is empty, then loop back to step #1. If there are not, then execute the signal engine’s update routine.

Figure 3 illustrates the structure of the interpreter’s loop, which threads the execution of the REPL and signal engine duties.

This achieves a perfectly-acceptable balance between minimal interruptions to the signal engine update loop and ensuring the responsiveness of the REPL in executing code sent by the user with as little latency as possible.

Of course, pausing the engine’s update loop to perform garbage collection and to execute any code that’s waiting in the buffer can occasionally lead to drops in the engine’s sample rate, but in our practical tests that never became an audibly perceptible—by virtue of the design of the REPL loop, the signal engine will resume its updating as soon as execution of the REPL expressions has finished, thus minimising the disruption. In addition, regularly performing the garbage collection at the beginning of each loop iteration instead of waiting for the waste to reach a certain threshold ensures that garbage collection will not take long enough that it becomes an audible artifact in the sample rate of the module’s produced signals.

Such issues might have been more relevant had we chosen to use the module to produce audio signals instead of control sequences, but our design goals were explicit in that we wanted to create a sequencer module. We therefore feel that this single-core implementation was a success.

Another challenge was the omission of Common Lisp’s macro systems from the *uLisp* implementation, mainly due to concerns around the implementation’s code size. We managed to work around this limitation by introducing ‘*faux-macros*’: regular functions which are registered with the interpreter in order to have their arguments automatically *quoted* (Lisp’s terminology for “do not *EVAL* this piece of code yet, instead keep it as an inert data structure”) when passed to the REPL, before being *EVAL*ed. This improves the user-experience of the DSL by removing the need to always remember to quote pieces of code that are always meant to be quoted, or understanding the concept of quotation at all.

Faux-macros are implemented as a ‘hack’ on the interpreter’s implementation: every time a piece of code is received by the REPL, a pre-processor will traverse it to see if any of the symbols in the function position (the first position of lists) match the list of known faux-macro symbols. If they do, it will modify the rest of the function call list by transforming each of the arguments to be quoted, by replacing them with a list containing the symbol *quote* followed by the rest of the argument. This implementation differs from real Lisp macros in that the function is still executed normally at runtime and returns regularly, versus being executed at compile-time and returning another piece of code to be evaluated at

runtime. For our purposes here, they are practically indistinguishable from the point of view of the user.

3.4 Interacting with *uSEQ*

The design choice of how the player livecodes the module was a challenging decision. Module-size screens seem restrictive, use up microprocessor resources, and add significant cost and complexity to the module's build. On the flip side, to control the module with a laptop doesn't address issues around ergonomic and ergodynamic dissonance, and barely feels different to livecoding with a typical setup.

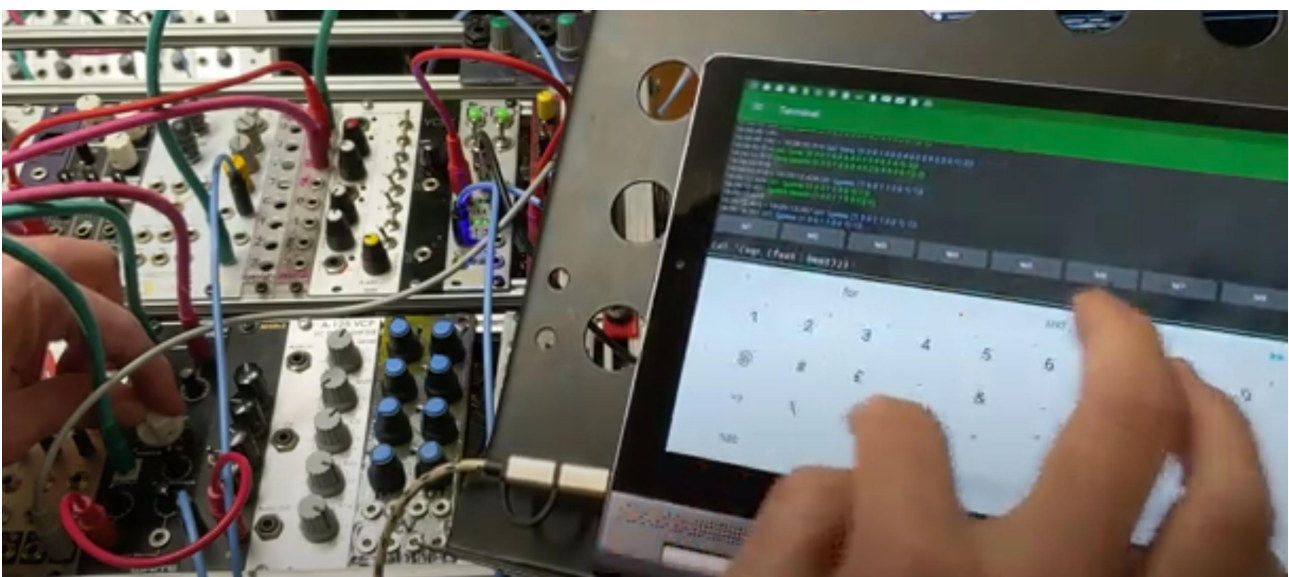
Our system chooses to make a compromise between these two extremes; the module offers a terminal connection for text interaction, using a modified serial-over-USB connection in the form of a stereo jack socket, to keep in the style of modular interfaces. The player can connect with any computing device with a serial terminal application. This could be a conventional computer, or also a mobile phone or tablet, or another module with a screen and keyboard.

This system leaves the choice to the user: they can customise the device and the software interface that they use to livecode. This system is also extendable; it's possible to run several modules at once, peppered throughout a rack as required, and plug in and out of them in turn to send new commands. The system could also be extended by opting to use the Raspberry Pi Pico W instead, which is WiFi-capable and can be controlled wirelessly from another mobile device.

In terms of physical distance between the livecoding instrument and the modular synthesiser, we found that the approach of using a tablet or other mobile devices was interesting in that it allowed the player to stay close to and focused on their modular case at all times. Of course, a trade-off between screen real estate and tactility of control has to be made when choosing a mobile device. A DIY solution to attach a tablet on the case itself could be easily devised, either with an attachable stand or with a custom-designed 3D printed accessory. We feel that this approach, in combination with a more advanced UI on the tablet that's designed specifically for livecoding on multitouch-capable screens, would be the most promising in terms of integrating the livecoding paradigm within the physical realm of the modular synthesiser instrument.

3.4.1 An Example of Use

We can illustrate the module with an example of use (shown in the accompanying video²). The module is sitting in a Eurorack system; *d1* is triggering and envelope generator, and *a2* is controlling the cutoff frequency of a filter. The module is being controlled from a serial USB terminal app on an Android tablet, mounted on a music stand next to the modular. The player is coding the module by editing saved text snippets in the terminal app, while also moving controls on the synthesiser. At the start of the demo, the filter is being controlled with a ramp waveform. Following this, the sequence is changed using a gate sequencer command and then the filter pattern is changed using an multistage envelope. This approach to configuration of text input and modular feels novel, and also flexible in the way that the instrument can be configured with varying computing devices and physical positioning (e.g. keyboard or touchscreen, embedded in the rack or free moving).



²<https://www.youtube.com/watch?v=dczVRTS2o0o>

3.4.2 Measuring Performance

In reasonable use (i.e. a pattern generator running on each of the 6 outputs), the runtime update loop takes between approximately 2ms and 10ms to execute, giving a sample rate of between 100Hz and 500Hz. For control signals, which is what this module is explicitly intended for, this sample rate is good and more than enough; an absolute minimum would be 20Hz. *uSEQ* takes around 6 or 7ms to read in and evaluate new commands, interrupting the flow of the runtime loop each time it does this because of the single core architecture of this first prototype. The adaptive timing system compensates for this interruption by resuming the engine's update routine as soon as evaluation of the incoming commands has finished. Overall, the system seems to be working within the bounds for satisfactory performance for musical use. If faster updates are required, the Raspberry Pi Pico can be overclocked from the default 133MHz up to 300MHz (although we have not tested this yet).

4 Conclusions

Returning to our original research question, we asked *how can we combine the flexibility of livecoding with the richness of modular synthesis in a useful way, by reducing this dissonance between systems?* We also stated the objective of making a low-cost, affordable and extensible instrument, in the spirit of open source culture in both livecoding and modular synthesis. Answering this latter objective, the module costs around £20 in parts to build, which is very cheap for a eurorack module of this functionality. It's also possible to 3D print and solder the module together as a hobbyist, although some of the build processes might be considered intermediate; some challenging soldering is required to connect the custom USB input to the pads on the underside of the Pico board, and the overall circuit is reasonably complicated (e.g. there are 27 wires to connect to the interface components). A PCB design would simplify construction but add to cost for low-volume productions.

The system is certainly extensible and customisable. The use of *uLisp* means that players can create customised libraries of livecoding routines, as well as hack on the core library's and interpreter's implementation itself to build an entirely new DSL. Our *faux-macros* hack, while not comparable to a full-blown Lisp macro system, was successful enough in enabling the further streamlining of the DSL, and is extensible by end users too. The combination of the general-purpose signal algebra and Lisp's ability to generate, modify, and evaluate code at runtime can lead to some particularly novel *time-varying metaprogramming*—function-of-time-generating code as a function of time *itself*.

The wider question is around how this module might reduce ergodynamic dissonance between livecoding and modular approaches. We can't give an objective answer to this question, although we can say that we have given the opportunity to move away from a conventional laptop livecoding environment, by enabling this avenue of customised serial input methods. One issue is that standard serial terminal apps are not very well suited for livecoding; some customisation would be needed for a better experience. However, the fact that you can customise the interface is promising. The *uLisp* signal engine attempts to work within a similar paradigm as modular, by treating everything as a signal. The player can make their own choice of screen/keyboard devices, as long as the device supports serial communication. Custom attachments for multiple tablets and/or mobile phones can be 3D printed (or otherwise fabricated) to bring the livecoding interface as physically close as possible to the rest of the modular system.

Future work could include the design of domain-specific terminal apps for touchscreen-enabled mobile devices, which could utilise the potential of the touchscreen medium to improve the experience of entering and modifying code, as well as to enable the application of continuous control over multiple parameters in the code through multitouch gestures. Additional inputs into the module could allow more extensive integration of signals generated within the modular system to control parts of the code, leading to more equally bidirectional communication between the two realms. Future versions of this module's firmware could see the full utilisation of the second core on the Pi Pico microcontroller, and future hardware versions could utilise more and more powerful microcontrollers that suitable for the generation of more and more complex continuous signals, for example for use as complex LFOs or envelopes.

In conclusion, we have presented *uSEQ*, a novel and DIY-friendly module for livecoding modular synthesisers, which attempts to integrate into the modular synthesis environment while leveraging the flexibility of a livecoding approach. We are just now exploring early versions, and hope to learn more about this approach through practice-led research in the future; our own and of others who wish to join us.

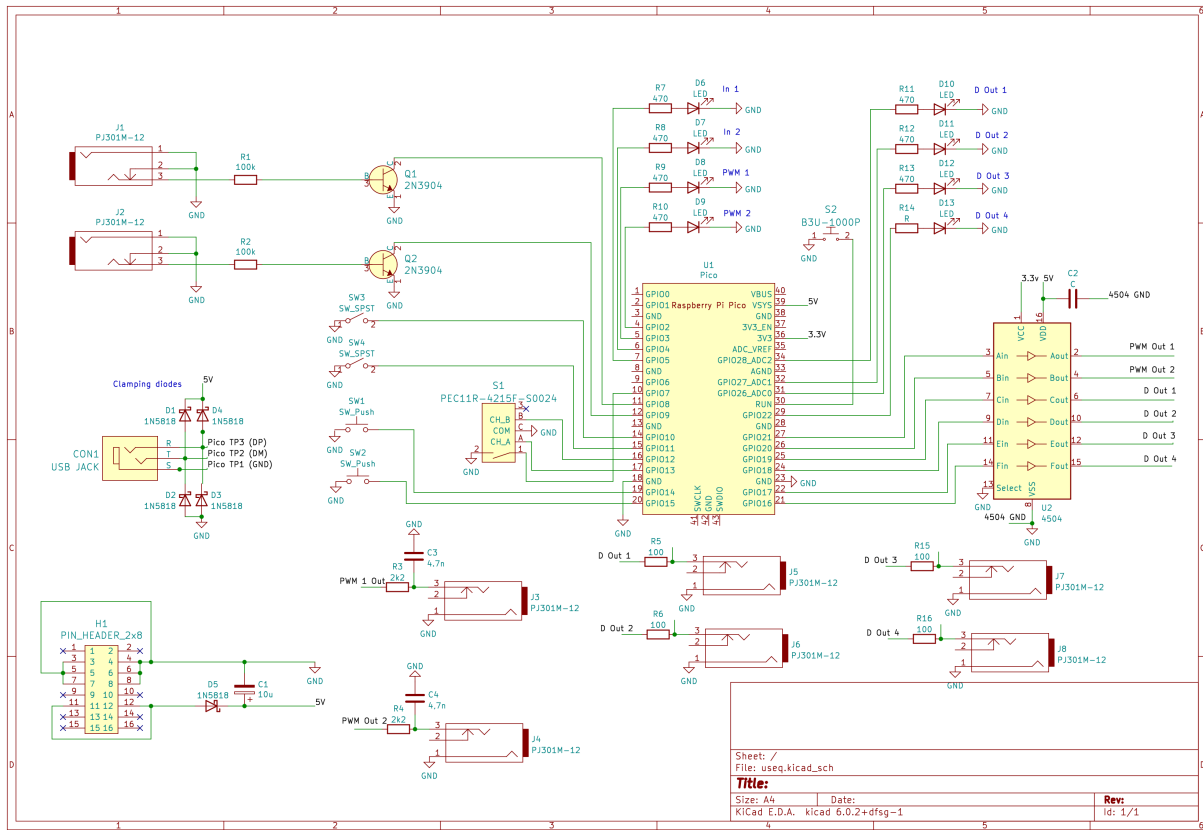
References

- 100rabbits. n.d. “Tools Ecosystem.” *100r.co*. https://100r.co/site/tools_ecosystem.html.
- Armstrong, Newton. 2006. *An Enactive Approach to Digital Musical Instrument Design*. Vol. 13. Princeton University.
- Bernardo, Francisco, Chris Kiefer, and Thor Magnusson. 2020. “A Signal Engine for a Live Coding Language Ecosystem.” *Journal of the Audio Engineering Society* 68 (10): 756–66.
- Drymonitis, Alexandros. 2022. *Live Coding on a Modular Synthesizer*. Zenodo. <https://doi.org/10.5281/zenodo.5888343>.
- Hutchins, Charles Celeste. 2015. “Live Patch/Live Code.” In *Proceedings of the First International Conference on Live Coding*, 147–51.
- Kiefer, Chris. 2018. “Between Code and Controllerism.” *Dancecult*.
- Kyriakoudis, Dimitris. 2018. “TimeLines: A Functional Reactive Approach to Live Coding Music as Functions of Explicit Time.” <https://github.com/InfiniteMonkeys/TimeLines-hs/blob/master/doc/Kyriakoudis2018.pdf>.
- Magnusson, Thor. 2019. *Sonic Writing: Technologies of Material, Symbolic, and Signal Inscriptions*. Bloomsbury Academic.
- McIlroy, M. Douglas. 1960. “Macro Instruction Extensions of Compiler Languages.” *Commun. ACM* 3 (4): 214–20. <https://doi.org/10.1145/367177.367223>.
- McKemie, Daniel. 2015. “Live Coding Control of a Modular Synthesizer with Chuck.” <https://toplap.org/2019/08/02/live-coding-control-of-a-modular-synthesizer-with-chuck/>.
- Mclean, Alex. 2014. “Making Programming Languages to Dance to: Live Coding with Tidal.” In *FARM 2014 - Proceedings of the 2014 ACM SIGPLAN International Workshop on Functional Art, Music, Modelling and Design*. <https://doi.org/10.1145/2633638.2633647>.
- Mellis, David A, and Leah Buechley. 2011. “Scaffolding Creativity with Open-Source Hardware.” In *Proceedings of the 8th ACM Conference on Creativity and Cognition*, 373–74.
- Monome. 2022. “Teletype.” <https://monome.org/docs/teletype/>.
- Norman, Donald. 1991. “Cognitive Artifacts.” In *Designing Interaction: Psychology at the Human-Computer Interface*, 333.
- Olofsson, Fredrik. 2015. “Chain Reaction.” <https://iclc.toplap.org/2015/performances.html>.
- Schafmeister, Christian. 2018. “2018 LLVM Developers’ Meeting: C. Schafmeister ‘Lessons Learned Implementing Common Lisp with LLVM’.” *Www.youtube.com*. <https://www.youtube.com/watch?v=mbdXeRBbgDM&t=632s>.

Appendix 1: Hardware Details

Hardware files are available at our github repository³.

Schematics



³<https://github.com/chriskiefer/ulisp-arm-livecode>

Bill of Materials

Qty	Reference(s)	Value
1	C1	10u
1	C2	100n
1	C3	4.7n
1	C4	4.7n
1	CON1	Stereo 3.5mm Jack
5	D1, D2, D3, D4, D5	1N5818
8	D6, D7, D8, D9, D10, D11, D12, D13	LED
1	H1	PIN_HEADER_2x8
2	J1, J2	PJ301M-12
6	J3, J4, J5, J6, J7, J8	PJ301M-12
2	Q1, Q2	2N3904
2	R1, R2	100k
2	R3, R4	2k2
4	R5, R6, R15, R16	100
7	R7, R8, R9, R10, R11, R12, R13	470
1	R14	470
1	S1	PEC11R-4215F-S0024
1	S2	B3U-1000P
2	SW1, SW2	SW_Push
2	SW3, SW4	SW_SPST
1	U1	Pi Pico
1	U2	CD4504BE